

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Проектирование защищённых интеллектуальных
информационных систем»
на тему:

Анализ защищённости приложений.

Часть 1. Оценка защищённости разработанного приложения.

Анализ защищённости программных систем.

Часть 2. Оценка защищённости сторонних программных систем

Студенты гр. 321701

И. А. Политыко

В. А. Лукашов

Руководитель

Д. А. Сальников

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Постановка задачи	4
2 Часть 1. Оценка защищённости разработанного приложения . .	5
3 Часть 2. Оценка защищённости сторонних программных систем	8
Заключение	10
Список использованных источников	11

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

CSRF	— Cross-Site Request Forgery, Межсайтовая подделка запроса;
XSS	— Cross-Site Scripting, Межсайтовый скриптинг;
SQL	— Structured Query Language;
ORM	— Object-Relational Mapping;
TLS	— Transport Layer Security;
PBKDF2	— Password-Based Key Derivation Function 2;
Fernet	— Симметричная схема шифрования (AES + HMAC) из библиотеки cryptography;
СУБД	— Система управления базами данных;
CRUD	— Create, Read, Update, Delete;
MitM	— Man-in-the-Middle, Атака «человек посередине».

1 ПОСТАНОВКА ЗАДАЧИ

В части 1 необходимо разработать приложение, позволяющее добавить пользователя в систему, авторизовать его по идентификационным данным, создавать, редактировать, удалять и искать конфиденциальные и неконфиденциальные данные, а также деавторизовать пользователя. Далее выполнить анализ защищённости с позиций политики безопасности ОС, компонентов и их взаимодействия, сетевой политики и конфиденциальности данных.

В части 2 необходимо разработать методику оценки защищённости, применить её к приложению из части 1, предложить комплекс мер усовершенствования и рекомендации по безопасному использованию.

2 ЧАСТЬ 1. ОЦЕНКА ЗАЩИЩЁННОСТИ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ

Краткое описание приложения. Разработано веб-приложение на языке Python с каркасом Flask и встроенной СУБД SQLite. Оно позволяет зарегистрировать пользователя, войти по паролю, выполнять создание, изменение, удаление и поиск двух классов записей и выйти из сессии. Неконфиденциальные заметки хранятся открытым текстом. Конфиденциальные секреты после ввода шифруются схемой Fernet: в базе остаётся шифртекст, на списке отображается только заголовок. Ключ шифрования и секрет приложения записываются в каталог экземпляра с правами только для владельца. Сервер по умолчанию принимает соединения на локальном интерфейсе. Поток данных показан на рисунке 2.1. Интерфейс — на рисунках 2.2 и 2.3.

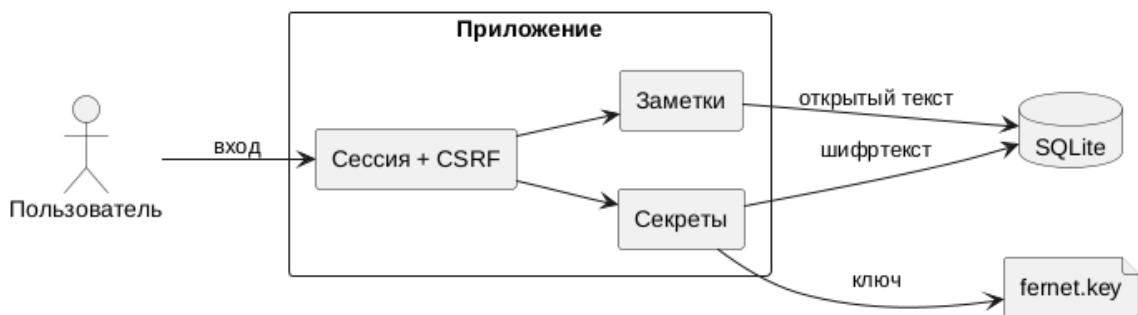


Рисунок 2.1 — Разделение заметок и секретов при записи в хранилище

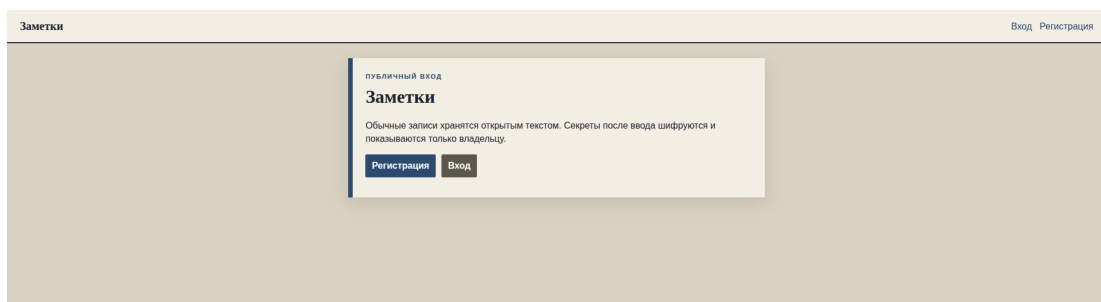


Рисунок 2.2 — Публичная страница и вход в систему

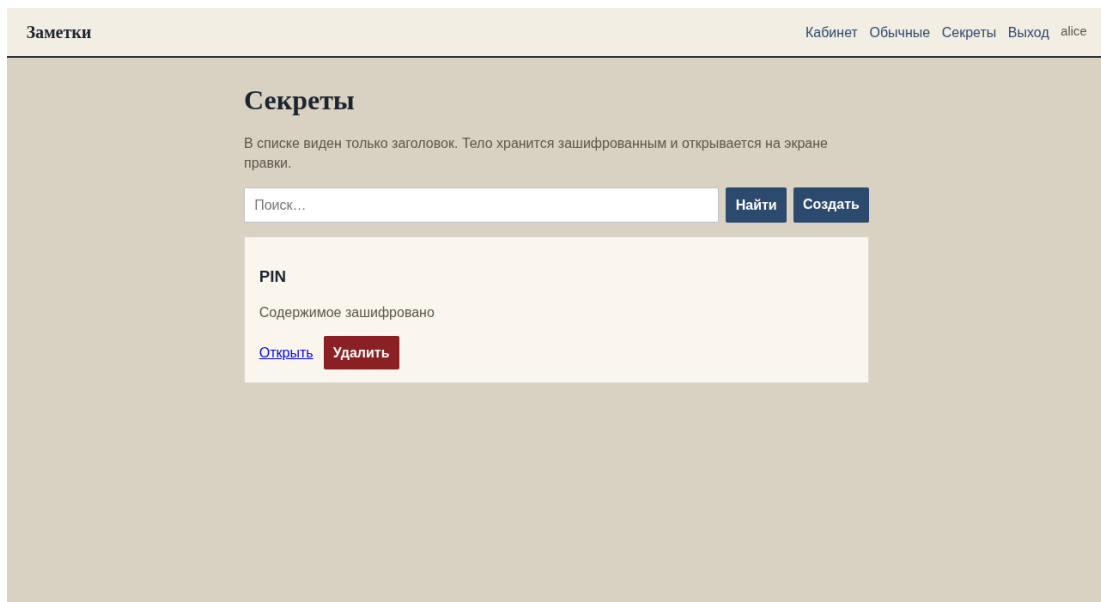


Рисунок 2.3— Список секретов без открытого текста тела

Запуск сервера: виртуальное окружение Python, установка зависимостей из файла требований, затем `python app.py`. Служба отвечает на `http://127.0.0.1:5050/`.

Анализ защищённости приложения. С позиции операционной системы процесс работает без прав суперпользователя. Каталог экземпляра имеет режим 700, файлы ключа и базы — 600. Это закрывает доступ другим учётным записям ОС, но не защищает от любого процесса того же пользователя: ключ лежит рядом с базой, и их совместное чтение даёт все секреты. Мера: отдельный системный пользователь и шифрование раздела.

С позиции компонентов и взаимодействия пароли хранятся как хэш PBKDF2-SHA256; изменяющие запросы проверяют CSRF-токен сессии (листинг 2.1); выборка записей фильтруется по идентификатору владельца (листинг 2.2); доступ к СУБД идёт через ORM; шаблоны экранируют вывод. На вход поставлено ограничение частоты неудачных попыток. Заголовки безопасности запрещают встраивание страницы в кадр и подключают CSP. Ограничения: один ключ Fernet на всё приложение; заголовки секретов остаются открытым текстом.

Сетевая политика: прослушивание только 127.0.0.1; cookie с флагами `HttpOnly` и `SameSite=Lax`; флаг `Secure` включается при работе через HTTPS. Без TLS вынос порта в сеть позволяет перехватить сессию.

Анализ защищённости конфиденциальной информации. К конфиденциальным отнесены пароли, тела секретов, ключ Fernet и идентификатор сессии; к неконфиденциальным — заголовки и тексты заметок владельца. Разделение сделано разными сущностями и разным способом хранения. Расшифрование тела выполняется только для владельца на экране правки. Поиск секретов идёт по заголовку и не вызывает расшифрование.

Ключ нельзя помещать в систему контроля версий.

Демонстрация защищённости. Практическая проверка: запрос к чужому идентификатору секрета возвращает отказ; POST без токена CSRF отклоняется; в SQLite заметка «Повестка» читается как текст, тело секрета — двоичный шифртекст, пароль — строка PBKDF2. Вид хранилища — таблица 2.1.

Поле	Значение
заметка	заголовок «Повестка», тело «встреча в 10:00»
секрет PIN	длина BLOB 100 байт, префикс hex 674141414141427170485246
alice	pbkdf2:sha256:1000000\$6umpCH...
bob	pbkdf2:sha256:1000000\$upbYko...
права	каталог экземпляра 0700; fernet.key, notes.db, flask.secret 0600

Таблица 2.1 — Вид записей в SQLite и режимы файлов экземпляра

```
1 @app.before_request
2 def _csrf_protect():
3     if request.method in ("POST", "PUT", "DELETE", "PATCH"):
4         token = session.get("_csrf")
5         sent = request.form.get("_csrf") or \
6             request.headers.get("X-CSRF-Token")
7         if not token or not sent or not secrets.compare_digest(
8             str(token), str(sent)
9         ):
10            abort(400, description="CSRF token missing or invalid")
```

Листинг 2.1 — Проверка CSRF-токена изменяющих запросов

```
1 secret = Secret.query.filter_by(
2     id=secret_id, user_id=current_user().id
3 ).first_or_404()
```

Листинг 2.2 — Выборка секрета только по идентификатору владельца

3 ЧАСТЬ 2. ОЦЕНКА ЗАЩИЩЁННОСТИ СТОРОННИХ ПРОГРАММНЫХ СИСТЕМ

Краткое описание системы. Объект оценки — приложение части 1. Состав: интерпретатор Python, каркас Flask, SQLite, библиотека cryptography (Fernet), cookie-сессия, локальный HTTP. Активы: учётные данные, тела секретов, ключ шифрования, заметки.

Методика. Методика следует комплексному перечню методички. Этапы: инвентаризация; модель угроз (перебор, словарь, MitM, инъекции, CSRF, чтение файлов хранилища); физические носители; ОС и окружение; исходный код; компоненты и протоколы; взаимодействие; организационные меры; нештатное использование; практическая проверка; выводы. Риск высокий — если секрет читается без привилегий ОС; средний — если нужен тот же пользователь ОС или открытый канал; низкий — если закрывается конфигурацией учебного контура. Чек-лист практической проверки: регистрация и вход; CRUD; поиск заметок по заголовку и телу и поиск секретов только по заголовку; выход; чужой идентификатор; запрос без CSRF; осмотр прав и содержимого файлов экземпляра.

Анализ по методике. Физический доступ к диску учебного узла раскрывает ключ и базу вместе — высокий риск при краже носителя без шифрования раздела. На уровне ОС права 600/700 снижают риск до среднего для прочих uid, но не для владельца процесса. В коде нет сырого SQL и нет вывода секрета в список; CSRF закрывает подделку запроса из другого сайта. Компоненты согласованы: ORM, PBKDF2, Fernet, localhost. Нештатное использование: повторный перебор пароля ограничивается по времени; без этого средний риск словаря. Организационно ключ нужно ротировать и не копировать вместе с резервной копией базы.

Сводка рисков — таблица 3.1.

Угроза	Уровень	Смягчение сейчас
чтение ключа и БД тем же uid ОС	высокий	права 600, не root
перебор пароля	средний	лимит неудачных входов
MitM при публикации порта	средний	bind на localhost
CSRF / XSS / SQLi	низкий	токен, autoescape, ORM
чужой идентификатор записи	низкий	фильтр по владельцу

Таблица 3.1 — Риски по методике оценки

Комплекс мер усовершенствования. Вынести ключ из каталога данных; считать per-user ключ из пароля (KDF) и оборачивать им DEK записи; публиковать только через обратный прокси с TLS и флагом Secure

у cookie; вести журнал обращений к секретам; регулярно обновлять зависимости и ротировать секреты приложения.

Рекомендации по безопасному использованию. Не публиковать учебный порт в Internet без прокси и TLS. Не включать каталог экземпляра в публичные репозитории. Использовать длинные уникальные пароли. Запускать от отдельного непривилегированного пользователя ОС. После демонстрации удалять или ротировать учебные ключ и базу.

ЗАКЛЮЧЕНИЕ

В лабораторной работе разработано приложение с обычными и конфиденциальными записями, выполнен анализ защищённости и построена методика, применённая к этой системе.

Семантическая составляющая. Защищённость складывается из мер на уровнях ОС, приложения, сети и данных. Разделение сущностей и шифрование тел секретов на диске повышают устойчивость к чтению файла базы, но не заменяют защиту ключа и канала.

Прагматическая составляющая. Получены воспроизводимый стенд, осмотр хранилища и методика с уровнями риска, позволяющие повторять оценку и формулировать меры: вынос ключа, TLS, отдельный пользователь ОС.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Ховард М., Лебланк Д. Защищённый код. — М. : Русская редакция, 2004. — 704 с.
- [2] Ховард М., Лебланк Д., Вьегга Дж. 24 смертных греха компьютерной безопасности. — СПб. : Питер, 2010. — 400 с.
- [3] Панасенко С.П. Алгоритмы шифрования. Специальный справочник. — СПб. : БХВ-Петербург, 2009. — 576 с.
- [4] CWE: Common Weakness Enumeration [Электронный ресурс]. — Режим доступа: <http://cwe.mitre.org/>.
- [5] OWASP Vulnerability Scanning Tools [Электронный ресурс]. — Режим доступа: https://owasp.org/www-community/Vulnerability_Scanning_Tools.
- [6] Flask documentation [Электронный ресурс]. — Режим доступа: <https://flask.palletsprojects.com/>.